

4. Hugging Face

Shashank Mohan Jain¹

(1) Bangalore, India

If you have even a passing familiarity with the advancements that have been made in the fields of machine learning and artificial intelligence in the years since 2018, you have almost certainly become aware of the tremendous strides that have been taken in the field of natural language processing (also known as NLP). Most of the progress in this area can be attributed to large language models, also known as LLMs. The architecture behind these LLMs is the transformer's encoder-decoder, which we discussed in Chapter 2.

Success of transformers came from the architecture's ability to process input data in parallel as well as having a better contextual understanding via the attention mechanism. We already referred to Vaswani's "Attention Is All You Need" paper in previous chapters. Before the emergence of transformers, the context was captured by vanilla RNN or LSTM without attention.

Hugging Face is now widely recognized as a one-stop shop for all things related to natural language processing (NLP), offering not only datasets and pretrained models but also a community and even a course.

Hugging Face is a new company that has been built on principles of using open source software and data. In true sense, the revolution in NLP started with democratization of the NLP models based on the transformer architecture. Hugging Face turned out to be a pioneer in not just open sourcing these models but also providing a handy and easy-to-use abstraction in the form of the Transformers library, which made it really easy to consume and infer these models.

Hugging Face provided a central place or hub for model developers to publish the models in the huggingface repository, which can then be consumed by consumers who are looking to build applications on top of these models. As an example, the BERT (Bidirectional Encoder Representations from Transformers) model was contributed by Google to huggingface. This then allowed a community of users to consume these models in their applications. Then came GPT models from OpenAI like GPT2, which are generative models and allowed the end user to write applications that can generate, say, stories, novels, etc. GPT2 is also a part of the huggingface ecosystem. Hugging Face provided not only APIs for consuming those models but also a way to fine-tune them with our own dataset and monitor and benchmark these models. So in a nutshell, the emergence of an ecosystem like huggingface has really opened up a plethora of opportunities for developers intending to build applications on top of natural language processing-based models.

Currently the scope of these models is not just limited to text processing. We now see the emergence of vision transformers and transformer-based models for audio. People are building applications for music generation or voice cloning in the audio domain and using them for fake image gen-

eration in the case of vision use cases. There are also models that have mined scientific literature and can be used for extracting knowledge from scientific journals. Similarly, models based on law-related documents have emerged, and people can use them to build a question and answering system on top of them. These models come in a variety of sizes based on the underlying architectures being used. As an example, the GPT3 model has around 175 billion parameters.

Similarly, the size and scope of their training datasets have both increased significantly. For instance, the original transformer was replaced by the much larger Transformer-XL ; the number of parameters in BERT-Base increased from 110 million to 340 million in Bert-Large; and the GPT2 model, which had 1.5 billion parameters, was replaced by the GPT3 model that has 175 billion parameters. China launched a model named Wu Dao 2.0, which has around 1.75 trillion parameters. The proponents of scaling opine that as we increase the sizes of these models, we will also be reaching our goal of artificial general intelligence (AGI) .

To give an example of infrastructure needed for such large models, GPT3 was trained on a super-computer with more than 10000 GPUs. This means training these models only lies in the realm of big companies. Now with the ability to consume most of these models, the end user becomes part of the application development ecosystem based around these large language models.

Features of the Hugging Face Platform

Because the Hugging Face platform is predicated on the idea of attention-based transformer models, it should come as no surprise that the Transformers library is at the center of the Hugging Face ecosystem. The accompanying Datasets and Tokenizers libraries offer assistance to the Transformers library. Keep in mind that transformers are unable to comprehend text in its original form, which is a string of characters. Since our inputs to transformers are in text, this text has to be encoded in a way that makes it consumable via the transformer-based neural network architecture . For this we make use of huggingface-provided APIs for tokenizing, which are known as tokenizers.

Apart from tokenizing, we might need to use some custom datasets to either fine-tune existing models or train the models from scratch. To have a uniformity in architecture, huggingface provides an abstraction for datasets via the Datasets API. The user can then have their own datasets, upload the datasets, train/fine-tune the models, and also upload the trained models all just by using the huggingface APIs. This is nothing less than revolutionary.

Components of Hugging Face

The huggingface library is based on a set of rich abstractions, which abstract out the complexity of creating applications based on natural language processing. These abstractions allow us a single interface to load models, use tokenizers, use datasets across a variety of models, tokenizers and datasets. This is really an empowering experience for the developer whose job is simplified by using these abstractions. We discuss some of these abstractions in the following subsections.

Pipelines

Pipelines provide a powerful and convenient abstraction for consuming the pretrained models from the huggingface model repository. It offers a straightforward application programming interface (API) that is dedicated to a variety of tasks:

1. Determine whether the overall sentiment of the sentence can be characterized as positive or negative.
2. Question and answering takes a question and pulls an answer out of the text that corresponds to it.
3. The masked language modeling technique suggests possible words to fill masked input with the given context.
4. The named entity recognition program will automatically assign a label to each of the tokens that are included in the input.
5. Reducing a longer piece of writing or an article into a more concise summary is referred to as summarization.

Pipelines are an abstraction built on top of three huggingface components, namely

1. Tokenizer
2. Model
3. Post-processor

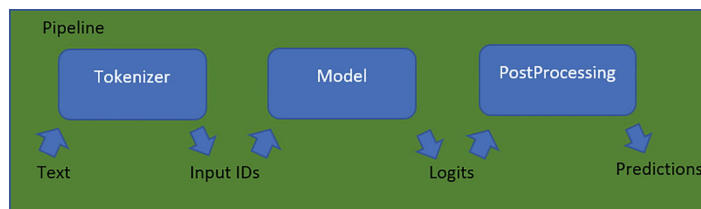


Figure 4-1 Workflow of a huggingface pipeline

Tokenizer

The first component in the pipeline is a tokenizer, which takes raw text as input and converts it into numbers so that the model can interpret them.

The tokenizer is responsible for

1. The process of separating the input into individual tokens, which may be words, sub-words, or symbols (such as punctuations)
2. Converting each token into an integer
3. Introducing new variables into the model that might prove to be of some use

When the model was trained, we had the need to tokenize the inputs. At that time there would have been a use of a certain tokenizer. We need to make sure that during usage of this model on actual inputs, we use the same tokenizer. This task is made easy by the `AutoTokenizer` class, which will automatically load the tokenizer used during the training. This makes the life simple for a developer considerably.

We load the tokenizer used to pretrain the GPT2 model in the following code.

First, install the Transformers library in Google Colab:

```
!pip install transformers
```

Figure 4-2 shows the installation process of the Transformers library from huggingface.

```
!pip install transformers

Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting transformers
  Downloading transformers-4.20.1-py3-none-any.whl (4.4 MB)
    4.4 MB 5.0 MB/s
Requirement already satisfied: filelock in /usr/local/lib/python3.7/dist-packages (from transformers) (3.7.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.7/dist-packages (from transformers) (1.21.6)
Requirement already satisfied: importlib-metadata in /usr/local/lib/python3.7/dist-packages (from transformers) (4.11.4)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.7/dist-packages (from transformers) (4.64.0)
Collecting tokenizers<0.11.3,0.13,=>0.11.1
  Downloading tokenizers-0.12.1-cp37-cp37m-manylinux_2_12_x86_64.whl (6.6 MB)
    6.6 MB 46.4 MB/s
Collecting pyyaml>=5.1
  Downloading PyYAML-6.0-cp37-cp37m-manylinux_2_5_x86_64.whl (641 kB)
    641 kB 33.8 MB/s
Collecting huggingface-hub<1.0,=>0.1.0
  Downloading huggingface-hub-0.8.1-py3-none-any.whl (101 kB)
    101 kB 9.2 MB/s
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.7/dist-packages (from transformers) (2022.6.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.7/dist-packages (from transformers) (21.3)
Requirement already satisfied: requests in /usr/local/lib/python3.7/dist-packages (from transformers) (2.23.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.7/dist-packages (from huggingface-hub<1.0,=>0.1.0) (4.1.1)
Requirement already satisfied: pyparsing>=3.0.5, >=2.0.2 in /usr/local/lib/python3.7/dist-packages (from packaging>=20.0->transformers) (3.0.9)
Requirement already satisfied: zipp>=0.5 in /usr/local/lib/python3.7/dist-packages (from importlib-metadata->transformers) (3.6.0)
Requirement already satisfied: charset>=3.0.2 in /usr/local/lib/python3.7/dist-packages (from requests->transformers) (3.7.4)
Requirement already satisfied: urllib3!=1.25.0,!=1.25.1,<1.26,>=1.21.1 in /usr/local/lib/python3.7/dist-packages (from requests->transformers) (1.26.13)
```

Figure 4-2 Installation of the Transformers library in Google Colab

Next, add this code.

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")
encoding = tokenizer("This is my first stab at AutoTokenizer")
print(encoding)
```

Listing 4-1 Code for a simple tokenizer

Executing Listing 4-1 in Google Colab results in the output shown in Figure 4-3.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

encoding = tokenizer("This is my first stab at AutoTokenizer")
print(encoding)

Downloading: 100% 665/665 [00:00<00:00, 3.89kB/s]
Downloading: 100% 0.99M/0.99M [00:00<00:00, 8.18kB/s]
Downloading: 100% 446k/446k [00:00<00:00, 8.78kB/s]
Downloading: 100% 1.29M/1.29M [00:00<00:00, 3.63MB/s]
{'input_ids': [1212, 318, 616, 717, 8903, 379, 11160, 30642, 7509], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1]}
```

Figure 4-3 Downloading of a tokenizer

The tokenizer will provide a dictionary with the following entries:

The numerical representations of your tokens are referred to as input IDs.

The attention mask is a mask that specifies which tokens need to have attention paid to them.

We can also pass multiple strings as inputs to the tokenizer. An example is shown in Listing 4-2, which tokenizes the text in two sentences.

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")
encoding = tokenizer("This is my first stab at AutoTokenizer", "life is what happens when you are p")
print(encoding)
```

Listing 4-2 Code for using a tokenizer for tokenizing text

This would give tokens for individual sentences as shown in Figure 4-4.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

encoding = tokenizer("This is my first stab at AutoTokenizer", "life is what happens when you are planning other things")
print(encoding)

{'input_ids': [1212, 318, 616, 717, 8383, 379, 11168, 38642, 7589, 6842, 318, 644, 4325, 618, 345, 389, 5418, 584, 1243], 'attention_mask': [1, 1,
```

Figure 4-4 Tokenized sentences

Instead of GPT2 we can also use other models like BERT. An example of a BERT-based tokenizer is given in Listing 4-3.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
encoding = tokenizer("This is my first stab at AutoTokenizer")
print(encoding)
```

Listing 4-3 Using BERT for tokenizing the text

Figure 4-5 shows the output of a BERT-based tokenizer. This is achieved by running Listing 4-3 in Google Colab.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

encoding = tokenizer("This is my first stab at AutoTokenizer")
print(encoding)
```

Downloading: 100% 29.0/29.0 [00:00<00:00, 572B/s]
 Downloading: 100% 570/570 [00:00<00:00, 10.7kB/s]
 Downloading: 100% 208k/208k [00:00<00:00, 2.09MB/s]
 Downloading: 100% 428k/428k [00:00<00:00, 2.64MB/s]
 {'input_ids': [101, 1188, 1110, 1139, 1148, 19428, 1120, 12983, 1942, 27443, 17260, 102], 'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]}

Figure 4-5 Execution of a BERT-based tokenizer

The output encoding is

```
{'input_ids': [101, 1188, 1110, 1139, 1148, 19428, 1120, 12983, 1942, 27443, 17260, 102], 'token_t
```

This returns a dictionary that contains the following three significant items:

- 1.The indices that correspond to each token in the sentence are denoted by the input ids variable.
- 2.The value of the attention mask specifies whether or not a token needs to be attended to.
- 3.When there is more than one sequence, the token type ids variable is used to determine which sequence a token is a part of.

We can get back the input by decoding the input_ids as shown in the following:

```
tokenizer.decode(encoding["input_ids"])
```

Its output is as shown in Figure 4-6.

```
▶ tokenizer.decode(encoding["input_ids"])

'[CLS] This is my first stab at AutoTokenizer [SEP]'
```

Figure 4-6 Shows the decoding process by taking tokens as inputs and returning the text as output

As can be seen, the tokenizer inserted two specialized tokens into the sentence. These tokens are known as CLS and SEP, which stand for classifier and separator, respectively. The tokenizer will take care of adding any necessary special tokens for you, provided that the model in question actually requires them.

Let's pass multiple sentences to this tokenizer as shown in Listing 4-4.

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
encoding = tokenizer("This is my first stab at AutoTokenizer", "life is what happens when you are p
print(encoding)
```

Listing 4-4 This code takes two sentences as input and generates tokens for them

This results in multiple encodings :

```
{'input_ids': [101, 1188, 1110, 1139, 1148, 19428, 1120, 12983, 1942, 27443, 17260, 102, 1297, 1116
```

Padding

When we process a group of sentences, their individual lengths do not always remain consistent. The inputs to the models need to have the same size as this is based on the underlying standard architecture. This presents a problem. The addition of a padding token to a sentence that contains an insufficient number of tokens is an example of the strategy known as “padding.”

Now we run the following example with multiple inputs with padding set to true as shown in Listing 4-5.

```
from transformers import AutoTokenizer
bert_tk = AutoTokenizer.from_pretrained("bert-base-cased")
sentences=["This is my first stab at AutoTokenizer", "life is what happens when you are planning otl
encoding = bert_tk(sentences, padding=True)
print(encoding)
```

Listing 4-5 This code shows how padding for tokenizers works

This gives the output as in the following:

```
{'input_ids': [[101, 1188, 1110, 1139, 1148, 19428, 1120, 12983, 1942, 27443, 17260, 102], [101, 11
```

As we can see, the third sentence was shorter in length, and thereby the tokenizer padded it with zeroes.

Truncation

It's possible that a model can't deal with a sequence that's too long sometimes. In this particular scenario, you will be required to condense the sequence down to a more manageable length.

If you want to truncate a sequence so that it is the maximum length that the model will accept, set the truncation parameter to true as shown in Listing 4-6.

```
from transformers import AutoTokenizer

bert_base_tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
sentences=["This is my first stab at AutoTokenizer", "life is what happens when you are planning otl
encoding = bert_base_tokenizer(sentences, padding=True, truncation=True)
print(encoding)
```

Listing 4-6 This code shows how the truncation flag

```
works in tokenizers
```

The output is shown in the following:

```
{'input_ids': [[101, 1188, 1110, 1139, 1148, 19428, 1120, 12983, 1942, 27443, 17260, 102, 0, 0, 0,
```

The next stage in the pipeline is the model.

AutoModel

We will check how AutoModel makes life simpler for us in terms of loading pretrained models.

The process of loading pretrained instances is made straightforward and unified by the Transformers library . This indicates that you are able to load an AutoModel in the same way that you load an AutoTokenizer . The sole distinction lies in selecting the appropriate AutoModel for the task at hand.

If we take an example of text classification , the way we load the model is shown in the following.

We will follow these steps:

- 1.Create an instance of a tokenizer and a model based on the name of the checkpoint. The model is determined to be a BERT model, and then weights that have been saved in the checkpoint are loaded into it.
- 2.Get the tokens and pass it to the model.
- 3.The model returns the logits.
- 4.Apply a softmax to calculate the probability of the class in which the sentence is classified (negative or positive for our following example).

Listing 4-7 is broken into multiple sections wherein Listing [4-7-1](#) explains how a tokenizer is loaded via the Transformers library .

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("siebert/sentiment-roberta-large-english")
sentences=["This is my first stab at AutoTokenizer","life is what happens when you are planning otl
encoding = tokenizer(sentences,padding=True,truncation=True,return_tensors="pt")
print(encoding)
```

Listing 4-7-1 Loading the tokenizer

This gives the following output:

```
{'input_ids': tensor([[ 0, 713, 16, 127, 78, 16735, 23, 8229, 45643, 6315,
2, 1, 1, 1, 1, 1, 1],
[ 0, 5367, 16, 99, 2594, 77, 47, 32, 1884, 97,
383, 4, 98, 563, 301, 14649, 2],
[ 0, 9226, 16, 45, 22307, 23, 1250, 2, 1, 1,
1, 1, 1, 1, 1, 1, 1]]), 'attention_mask': tensor([[1, 1, 1, 1,
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]])})
```

Listing [4-7-2](#) shows how a model is loaded via the Transformers library .

for loading the model

```
from transformers import AutoModelForSequenceClassification
model_name = "siebert/sentiment-roberta-large-english"
pt_model = AutoModelForSequenceClassification.from_pretrained(model_name)
```

Listing 4-7-2 Loading the model

```
#For printing the encoding
pt_outputs = pt_model(**encoding)
print (pt_outputs)
SequenceClassifierOutput(loss=None, logits=tensor([[ 3.0351, -2.1955], [-3.6225,  2.7819], [ 3.9581, -3.6334]], grad_fn=AddmmBackward0))
```

Listing 4-7-3 Generate the logits

#For printing the logits

```
logits=pt_outputs.logits
print (logits)
```

This outputs

```
tensor([[ 3.0351, -2.1955],
        [-3.6225,  2.7819],
        [ 3.9581, -3.6334]], grad_fn=AddmmBackward0))
```

Finally, we do a softmax to print the output probabilities as illustrated in Listing [4-7-4](#).

```
output = torch.softmax(logits, dim=1).tolist()[1]
print(output)
```

Listing 4-7-4 Print probabilities

specific to the individual class of sentiment

We get the following output:

```
[[0.9946781396865845, 0.005321894306689501], [0.001651538535952568, 0.9983484745025635], [0.9994951, 0.0005048484848484848]]
```

We can see the output probabilities for all three sentences:

```
This is my first stab at AutoTokenizer
```

The first column gives the probability of sentiment being negative and the second column of sentiment being positive:

```
Score [0.9946781396865845, 0.005321894306689501]
```

This reflects a negative sentiment in the sentence.

Next is for the second sentence:

```
life is what happens when you are planning other things. so plan life accordingly
[0.001651538535952568, 0.9983484745025635]
```

This reflects a positive sentiment in the sentence.

Now we look into a wrapper class known as pipeline , which can be used to achieve the same task with less code as shown in Listing 4-8.

```
from transformers import pipeline
# create a pipeline instance with a tokenizer and model
roberta_pipe = pipeline(
    "sentiment-analysis",
    model="siebert/sentiment-roberta-large-english",
    tokenizer="siebert/sentiment-roberta-large-english",
    return_all_scores = True
)
# analyze the sentiment for the 3 sentences we used in the preceding example
roberta_pipe(sentences)
```

Listing 4-8 This code shows how to use the pipeline API for doing sentiment analysis

We get the following output:

```
[{'label': 'NEGATIVE', 'score': 0.9946781396865845}, {'label': 'POSITIVE', 'score': 0.005321894306},
{'label': 'NEGATIVE', 'score': 0.001651539234444499}, {'label': 'POSITIVE', 'score': 0.998348474501},
{'label': 'NEGATIVE', 'score': 0.9994955062866211}, {'label': 'POSITIVE', 'score': 0.0005045001744}]
```

We can check that the outputs match when we didn't use and used the pipeline class in our code.

Summary

In this chapter we discussed the architecture of the huggingface library and its components like tokenizers and models. We also learned how we can use these components to do a simple task like analyzing the sentiments of the sentences. In the next chapter, we will take more examples of doing different kinds of tasks using the Transformers library .